Expresiones Regulares

Una **“expresión regular”** es describir un lenguaje con un patrón de texto. Al igual que los autómatas nos sirven para validar la aceptación de cadenas. Por lo que podemos considerar una expresión regular como otra manera de definir los Autómatas.

Por ello:

Expresión Regular ⇄ Autómata Finito

En resumen una expresión regular es una fórmula que describe un patrón de texto.

¿Para que se usan las expresiones regulares?

El uso de las expresiones regulares es muy amplio e incluso contiene material demasiado difícil de comprender que por limitaciones del intelecto del escritor no intentaremos profundizar. No quiero asustar al lector ya que el 99.9% del tiempo su función será solo de validar si un texto es correcto o no. Pero para citar algunos ejemplos de para qué sirve una expresión regular veamos esta mini lista:

- Validación de texto en formularios… correo correcto, teléfono correcto…
- Búsqueda de texto.
- Limpieza de datos.
- Análisis de LOGs
- Compiladores de lenguaje.
- Procesadores de texto.
- Validación de palindromo.
- Búsqueda de patrones en cadenas de ADN.
- …

Recordemos Autómatas y demos un ejemplo de expresión regular:

Supóngase que se necesita una expresión regular la cual reconoce si una cadena de texto contiene una “a”. Nosotros ya habíamos visto una manera de resolver este problema mediante los autómatas finitos.

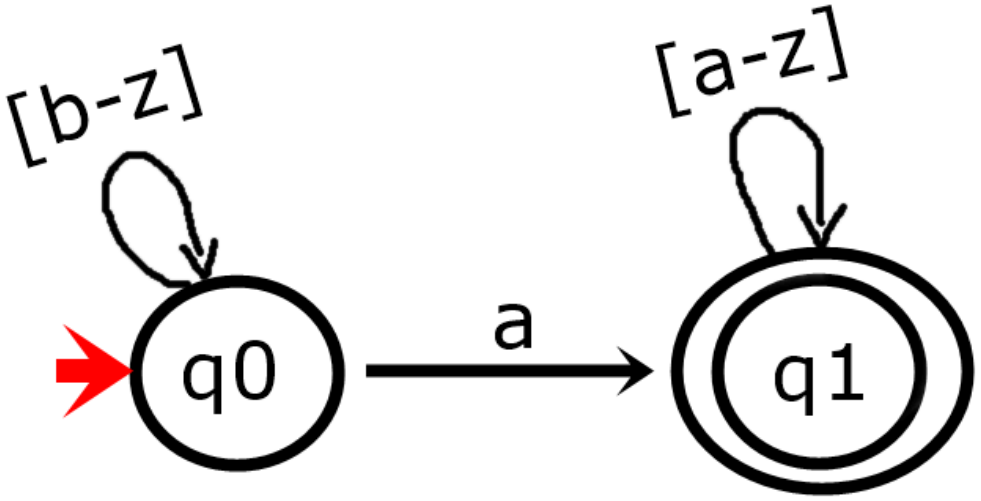

FIG 00: Autómata finito que reconoce si una palabra contiene la letra a.

A este punto el lector debe de reconocer que cualquier cadena de texto que contenga una “a” va a hacer que el autómata quede en el estado final “q1”... ahora bien… el concepto de expresión regular tiene que ver exactamente con el autómata que podemos observar… ya que una expresión regular que hace lo mismo del autómata anterior es solo:

.*a.*

Pero el lector podrá notar que está claro que contiene una a… no sabe que significa el punto o el asterisco… por eso nosotros lo primero que tenemos que entender son las bases de este lenguaje… osea como se construye…

Sintaxis de las expresiones regulares

Antes de aprender un nuevo idioma nosotros tenemos que conocer:

- sintaxis
- Operadores básicos.
- significado.

Recuerda… para aprender un nuevo idioma primero aprendemos vocabulario, gramática y por último las frases.

Que es una expresión regular

Es una fórmula/receta la cual se usa para reconocer/generar texto.

Caracteres en expresiones regulares

En una expresión regular los caracteres son los que nosotros usamos para el lenguaje, letras, números y símbolos de puntuación. Estos representan exactamente el carácter que se quiere reconocer.

Metacaracteres en expresiones regulares

Son símbolos con un significado especial… por lo general lo utilizamos para operaciones algunos ejemplos son:

*, +, ?, |, (,), [,], {, }, ., ^, $

Operaciones básicas en expresiones regulares

Las operaciones que se realizan en expresiones regulares son similares a las de los autómatas:

- Concatenación: Es escribir un símbolo después de otro.
- Unión: Para indicar si es esto o lo otro.
- Repetición: Indica cuántas veces puede repetirse.

tabla de resumen de expresiones, operaciones.

Ejemplo	***Significado***	***Descripción***
ε	Cadena vacía.	Representa al vacío… la cadena vacía la cual no contiene nada adentro. Ejemplo de aceptación: “”
^	Inicio.	Indica que una cadena va a comenzar. Ejemplo: ^Hola
a	Solo un carácter cualquiera	Es solo una expresión regular que acepta explícitamente la cadena “a”

ab	Concatenación de caracteres	Es una expresión regular que concatena 2 símbolos… y solo acepta la cadena “ab”.
a|b	Un carácter o el otro.	Esta expresión dice que solo acepta “a” o “b”
a*	Cualquier cantidad de a.	Esta operación lo que hace es aceptar una cadena con cualquier cantidad de “a” OJO: Puede ser una o muchas… o ninguna. Ejemplos de aceptación: “”, “a”, “aa”, … “aaaaaaaa”
a+	Un carácter o más.	Esta operación lo que hace es aceptar la cadena con 1 o más coincidencias: OJO: No puede ser vacía: Ejemplos de aceptación: “a”, “aa”, … “aaaaaaaa”
a?	OPCIONAL. Cero o una	Esta operación lo que hace es aceptar ninguna o una… OJO: No puede ser más de 1. Ejemplos de aceptación: “” o “a”
[abc]	Grupo de caracteres.	Esta cadena indica que hay una agrupación de 3 caracteres pero no son aceptados todos a la vez… solo es aceptado 1 de la lista. Ejemplos de aceptación: “a” o “b” o “c”
[a-z]	Rango de caracteres	Esta cadena indica que cualquier carácter contenido en el rango de la “a” a la “z” osea el abecedario es acepto… Ejemplos de aceptación: “a”, “b” … “z” Nota: Sólo acepta cualquier letra del abecedario en minúscula.
[^a]	Cualquier carácter excepto.	Es la negación… osea que no esté. Ejemplos de aceptación: “0”, …”9”, “b”, “Z”
.*	Cualquier cadena.	Acepta cualquier carácter… incluso cualquier cadena. Ejemplos de aceptación:

		“”, “a”, “felipe”, “123”
a{#}	Cuantificador	Esta cadena contiene la cantidad de veces que puede repetirse. Para estos casos se puede indicar solo 1 número para establecer una longitud fija o incluso indicar con un rango de números. Ejemplo de longitud fija: a{3} … acepta “aaa”. Ejemplo de longitud en rango: a{2,5} … acepta “aa”, “aaa”, “aaaa”, “aaaaa” Ejemplo de longitud mínima: a{3,} … acepta “aaa”, “aaaa” … “aaaaaaaaaa”
(ab)*	Agrupación	Esta cadena es capaz de unir bloques para operar. Acepta repeticiones de los caracteres las veces que sea. Ejemplo de aceptación: “”, “ab”, “abab”... “abababababab”
$	fin	Indica que una cadena ha terminado. Ejemplo: Hola$

Ahora antes de continuar vamos a conocer algunas de las expresiones regulares más utilizadas:

correo electrónico: ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$
número: ^[0-9]+$
palabra: ^[a-zA-Z]+$
Advertencia: es solo para conocimiento del lector… En capítulos futuros profundizaremos en cada una de ellas.

Hablemos sobre la cadena vacía

La cadena vacía es una de las más fundamentales en teoría de lenguajes, autómatas y expresiones regulares. Si lo entiendes podrás comprender “lenguajes formales”.

En resumen:

“La cadena vacía es la que no contiene ningún símbolo”

La representamos con el símbolo ε y es la cadena de longitud cero.

$$\varepsilon = ""$$

Cuidado con confundir con la nada… no es lo mismo el vacío que la nada

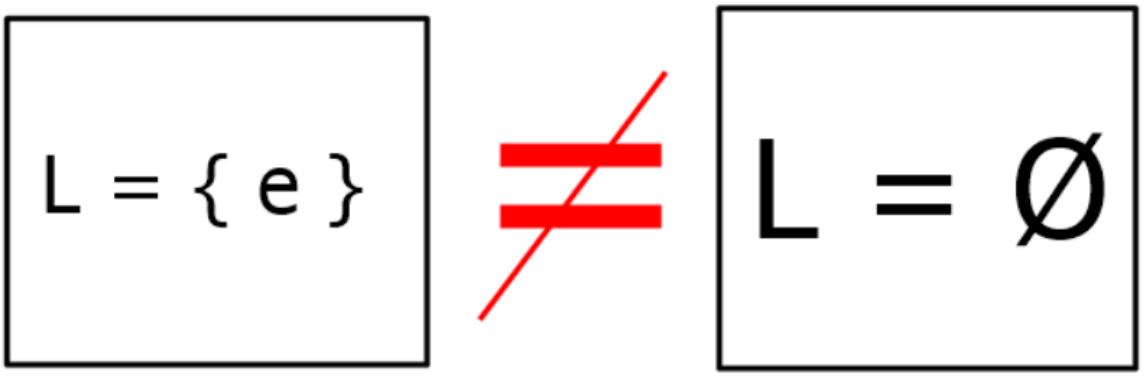

FIG 01: Comparativa del Vacío vs La Nada.

La cadena vacía es importante en “compiladores”, “validaciones”, “gramáticas” y la “teoría de lenguajes”.

Comparativa entre Autómata y Expresión Regular

Ahora que dominamos las reglas básicas del lenguaje vamos a comparar nuestra expresión regular “.*a.*” contra el autómata de la FIG 00.

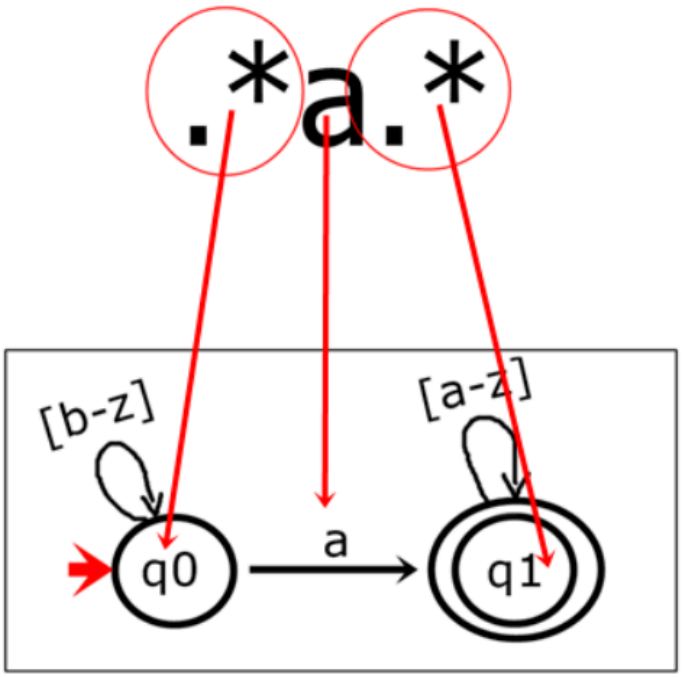

FIG 02: Comparativa Expresión Regular a Autómata.

Si el lector analiza esta imagen podrá ver una correlación entre el autómata y la expresión regular… osea es posible convertir de autómata a expresión regular y viceversa.

OJO: Actualmente el cerebro del lector debe ver que existe una dicotomía entre el (.*) ya que al principio de este capítulo se indicó que acepta cualquier cadena. desde el vacío y un rango de caracteres de la “a” a la “z”... pero en realidad la primera vez es “TODO DESDE LA “b” hasta la “z”” En el autómata… y luego de una aceptación “a” en realidad si es una cadena que acepta cualquier carácter. Esta dicotomía la trataremos en capítulos posteriores.

La dicotomía no contradice el concepto de .* Dado que una expresión regular y un autómata son equivalentes su funcionamiento interno es diferente… pero el resultado es el mismo… es como alcanzar un mismo objetivo mediante diferentes métodos.

Nuestro autómata acepta cualquier cadena que contenga una a y nuestra expresión regular también. Lo que cambia es el contexto de la expresión completa.

.* significa cualquier cadena.

.*a significa cualquier cadena y que contenga la “a”

.*a.* significa cualquier cadena que contenga “a”

Uso de expresiones Regulares en Python

```
import re
```

No vamos a construir un motor de expresiones regulares como lo hicimos con los autómatas… nosotros vamos a utilizar la librería ***re*** de Python… y esto es por que de momento estamos en un curso introductorio… pero en un futuro vamos a profundizar.

Ejercicio 01:

Implementar una expresión regular en python que sea capaz de reconocer si una palabra contiene la letra “a”

0 Introducción > 0.3-Regex > main.py > ...

```python
import re

# match str to contains 'a'
words = ["andrés", "michell", "felipe", "juanita"]

_ptrn = r".*a.*"

for itterW in words:
    result = re.match(_ptrn, itterW)
    print(f"La palabra {itterW} contiene a? >> RTA: {result}")
```

Ejercicio 02:

Se necesita validar si una entrada de texto que contiene el número de documento de una persona está compuesta solo por números.

```python
# match if number is valid
_cc = "1059785517"
_ptrn = r"[0-9]+"
result = re.match(_ptrn, _cc)

print(f"El documento {_cc} es valido? >> RTA: {result}")
```

Ejercicio 03:

Se necesita validar si una entrada de texto está en mayúsculas.

```python
# Validates if text is in upper case.
_txts = ["LILIANA", "hogar", "MADRE"]

_ptrn = r"^[A-Z]+$"

for i in _txts:
    result = re.match(_ptrn, i)
    print(f"La palabra {i} está en mayuscula? >> RTA: {result}")
```